Launch Manifest

THE AUTOWARE FOUNDATION

Declaring and Auditing the ROS 2 Communication Graph

play_launch — Phase 31

國立臺灣大學

NEWS Lab 嵌入式系統暨無線網路實驗室

Manifest in 30 Seconds

```yaml
version: 1

nodes:
  talker:
    pub: [chatter]
  listener:
    sub: [chatter]

topics:
  chatter:
    type: std_msgs/msg/String
    pub: [talker/chatter]
    sub: [listener/chatter]

exports:
  output: [talker/chatter]
```

One manifest per launch file. Topics wire endpoints. Names are relative.

Manifest Structure

nodes: endpoint declarations (pre-remap names)

topics: wiring + type + QoS + channel contract

includes: child scopes (name = namespace, manifest path)

imports: subscriber dependencies (what we need from outside)

exports: publisher outputs (what we provide to parent)

io: scope-level timing contract

Wiring: Topics Connect Endpoints

```yaml
nodes:
  cropbox_filter:
    pub: [output]       # pre-remap name
    sub: [input]

topics:
  cropped:              # → <ns>/cropped
    type: PointCloud2
    pub: [cropbox_filter/output]
    sub: [ground_filter/input]
    contract:
      rate_hz: 10
```

- **Endpoint names** = node's internal topic names (before <remap>)
- **Topic** gives the real name and wires pub ↔ sub
- **contract:** = channel-level quality bound (rate, deadline, drops)

Composition: Includes and Scope Hierarchy

```yaml
includes:
  lidar:                          # name = ROS namespace
    manifest: tier4_.../lidar_perception.launch.yaml
  camera:
    manifest: tier4_.../camera_perception.launch.yaml

topics:
  lidar_objects:
    pub: [lidar/output]           # child's export
    sub: [fusion_node/lidar_in]

imports:
  input: [lidar/input, camera/input]
exports:
  output: [tracker/tracked]
```

Parent wires children explicitly via topics. No implicit accumulation.

I/O Patterns in Autoware

Pipe (1→1)

on_arrival — cropbox, centerpoint

Fusion (N→1)

all_ready + timestamp — concat_pointclouds

Source (0→N)

periodic — lidar_driver (10 Hz)

Timer-driven

periodic + state inputs — tracker (10 Hz)

Feedback cycle

on_arrival + state — NDT ↔ EKF

All from the same primitives — no enumerated pattern types.

Node I/O Contract

```yaml
# Simple pipe
centerpoint:
  io: { latency_ms: 30 }

# Fusion — all inputs synchronized
fusion_node:
  io:
    trigger: all_ready
    correlation: timestamp
    tolerance_ms: 50
    latency_ms: 20

# Timer-driven with state inputs
tracker:
  io:
    state: [fused]
    trigger: { periodic: 10 }
    freshness_ms: 200
```

`io:` only adds annotations to **`sub:/pub:`**

No redundancy — timing and roles are separate from topology.

Contract Primitives

Input roles (per sub: endpoint)

role: trigger (default) — fires computation
role: state — read-latest when triggered
consume: N — messages per firing
min_count: N — readiness precondition

Trigger

on_arrival — any trigger-input (default)
all_ready — barrier, all must arrive
periodic: N — timer at N Hz

Correlation (2+ trigger inputs)

timestamp — match by header.stamp
latest — most recent of each

Timing bounds

latency_ms — trigger → output
freshness_ms — max input data age

Drop tolerance

max_consecutive_drops
max_drop_ratio over window

Topic Channel Contract

```yaml
topics:
  detected_objects:
    type: DetectedObjects
    pub: [centerpoint/objects]
    sub: [tracker/input]
    contract:
      rate_hz: 10
      deadline_ms: 150
      max_drop_ratio: 0.05
```

Two separate concerns:

- `Node io:` computation — how fast input → output
- `Topic contract:` channel — what the link promises

Full Pipeline: Top-Level Manifest

```yaml
includes:
  sensing:
    manifest: tier4_.../sensing.launch.yaml
  perception:
    manifest: tier4_.../perception.launch.yaml
  planning:
    manifest: tier4_.../planning.launch.yaml

topics:
  pointcloud:
    pub: [sensing/pointcloud]
    sub: [perception/input]
  tracked_objects:
    pub: [perception/output]
    sub: [planning/input]
```

Scope budgets:

sensing: source, 10 Hz
perception: 85 ms
  lidar: 50 ms
  camera: 30 ms
planning: 100 ms

Critical path:

max(50,30) + 20 + 100
= 170 ms

Working with Launch Files

- Manifest loaded alongside <include> — parser resolves namespace
- Endpoint names match <remap from="..."> in launch XML
  - Warn if remap is missing
- Same manifest reused across different namespace contexts
- Partial coverage: missing manifest = no auditing for that scope
- Full ROS 2 compatibility — manifests are sidecar files, not replacements

Contract Violation Detection via RCL Interception

Captured per message:

`mono_time`	wall-clock when pub/take happened
`header.stamp`	data timestamp from message
`topic_name`	which topic
`node_name`	which node

Output:

- `frontier_summary.json`
- `stats_summary.json`
- `manifest_audit.json`

→ contract violation checks

What the Monitor Checks

Node io: checks

latency_ms: pub_time - take_time ≤ bound
freshness_ms: pub_time - stamp ≤ bound

Topic contract: checks

rate_hz: interval ≈ 1/rate
deadline_ms: interval ≤ deadline
max_drop_ratio: drops/total ≤ ratio

Assumption	Guarantee	Diagnosis
OK	OK	Nominal
OK	FAIL	Node bug
FAIL	OK	Node robust
FAIL	FAIL	Upstream problem

Scope Budget Verification

- Critical path = longest path in DAG → O(V+E)
- scope.io.latency_ms ≥ longest_path(imports → exports)
- Composition rules:
  - Series: sum of latencies
  - Parallel: max of branches
  - Periodic: period + jitter
- Checked statically at parse time — before any node starts
- Algorithm: topological sort + dynamic programming (petgraph)